COMP 441 ? SOFTWARE ANALYSIS AND TESTING

Week 7 evidence: Mutation Testing Report v1 (Group A, Track D, tomlkit @ 8c959b5)

Group A ? Track D ? tomlkit

Results (AI-only suite `tests_ai/`, mutmut 3.8.0)

	Mutants
Killed	3,087
Timeout (counted as detected)	26
Survived	1,890
No tests (code never executed)	386
Total	**5,389**

Mutation score = (killed + timeout) / total = **57.8%** (62.2% of mutants in executed code). The AI-only suite and manual survivor tests pass together: **404 passed, 3 xfailed**.

Reproduce

```
. .venv/bin/activate && pip install mutmut==3.8.0 hypothesis
mutmut run --max-children 8          # config in pyproject.toml [tool.mutmut]; ~20 min on WSL/Python
3.14
mutmut results --all true > week7/mutmut_results_v1.txt
mutmut show <mutant-name>            # diff of one mutant

# kill tests written for 11 survivors (each fails against its mutant):
cp -r tests_manual mutants/ && cd mutants
MUTANT_UNDER_TEST="tomlkit.items.x‖Array‖insert__mutmut_14" python -m pytest tests_manual -k test_k6_

# risk-based prioritisation + APFD evaluation
pytest -q --cov=tomlkit --cov-context=test --cov-report= tests_ai --durations=0 >
week7/durations_ai.txt
pytest --collect-only -q tests_ai | grep :: > week7/collection_order.txt
python week7/scripts/kill_matrix.py mutants week7/mutmut_results_v1.txt 300 week7/kill_matrix.json
python week7/scripts/prioritise.py .coverage week7/radon_cc.json week7/churn_2y.tsv \
       week7/durations_ai.txt week7/kill_matrix.json week7
python week7/scripts/classify_survivors.py <survivor-list> <diff-dir> week7/survivor_categories_v1.csv
```

Survivor root-cause analysis

These eight cases show why the AI-only tests missed representative surviving mutants and how the manual tests target the gaps. The tests are in `tests_manual/test_kill_v1.py`.

Mutant	Why the AI-only tests missed it	Manual test and design technique
`InvalidNumberError.__init__` mutmut 3	Checked the exception type and position, but not the exact, case-sensitive message.	`test_k1_invalid_number_message_and_position_exact` checks the complete message and location (equivalence partitioning).
`Source._to_linecol` mutmut 7	Did not exercise the exact index at a newline boundary.	`test_k2_line_col_at_line_boundary` checks the character before and at the newline (boundary value analysis).
`Parser._parse_escaped_char` mutmut 1	Covered an invalid letter escape, not whitespace after a backslash in a single-line string.	`test_k3_backslash_space_rejected_in_single_line_string` checks rejection, alongside a multiline continuation control (equivalence partitioning).
`parse_rfc3339` mutmut 66	Used timezone offsets below ten hours, missing the two-digit-hour boundary.	`test_k4_two_digit_utc_offsets` checks `+10:00`, `-11:30`, and `+23:59` (boundary value analysis).

Mutant	Why the AI-only tests missed it	Manual test and design technique
`parse_rfc3339` mutmut 163	Checked fractional-second truncation for datetimes, but not the separate time-only parsing branch.	`test_k5_time_fraction_truncated_to_microseconds` checks both time-only parsing paths (equivalence partitioning).
`Array.insert` mutmut 14	Tested positive and negative insertion positions, but not index zero.	`test_k6_insert_at_index_zero` checks the boundary position, resulting values, and round-trip serialization (boundary value analysis).
`Array.__delitem__` mutmut 13	Did not cover stepped slices, a distinct slice behavior from contiguous deletion.	`test_k7_delete_stepped_slice` checks both even- and odd-index deletion (equivalence partitioning).
`Array.add_line` mutmut 8	Tested LF, but not CR or CRLF line breaks.	`test_k8_add_line_comment_rejects_any_line_break` checks all three line-ending forms (equivalence partitioning).

The extra strict xfail in the index-zero insertion tests records a separate spacing defect in the unmutated library; it is not counted as a mutation kill. All 11 targeted manual tests were also run against their named mutants; each mutant was killed.

Test-priority rationale

The risk-based order ranks tests by the additional complexity-weighted, two-year-churn risk they cover; shorter runtime breaks ties. Mutation outcomes are excluded when constructing the order, keeping the APFD comparison out-of-sample. The top-ranked parse test covers the most new risk, especially `Container.append` (complexity 74); rank 2 targets `Container._replace_at` (41), and rank 4 exercises `InlineTable.as_string` (30). This puts high-risk container and formatting behavior early without tuning the ranking to the sampled mutants.

The 300-mutant kill-matrix sample had one mutant that did not remain killed in the fresh v1 run; it was excluded. Of the remaining sample, 258 mutants had matching test identifiers for APFD. The risk-based order achieved APFD **0.7722**, compared with **0.6369** for collection order and **0.6911** for the mean of 200 shuffled orders. It detected half of those mutants within 16 tests, versus 51 in collection order. The full ranking and evaluation are in `priority_order.csv` and `prioritisation_eval.json`.

Files

- `mutmut_results_v1.txt`, `mutation_v1_summary.json` - every mutant's status; score per module
- `survivor_categories_v1.csv` - all 1,890 survivors with diff and heuristic category
- `kill_matrix.json` - which AI tests fail for a seeded sample of 300 mutants; one stale mutant is excluded from APFD
- `priority_order.csv` - ranked execution order for the suite

- `prioritisation_eval.json`, `detection_curve.csv` - APFD of risk-based vs pytest order vs random
- `churn_2y.tsv` - lines changed per file since 2024-10-18 (git log --numstat)